Ligero 구현 분석

개요

libiop 라이브러리의 Ligero 구현은 `ligero_iop_parameters` 클래스를 중심으로 한 매개변수 최적화 시스템과 `interleaved_r1cs_protocol` 클래스를 통한 실제 프로토콜 실행으로 구성됩니다. 이 문서에서는 핵심 구현 세부사항과 최적화 기법들을 분석합니다.

ligero_iop_parameters 클래스 분석

1. 클래스 구조와 핵심 매개변수

```
template<typename FieldT>
class ligero_iop_parameters {
protected:
std::size_t security_parameter_; // 보안 매개변수 (λ)
LDT_reducer_soundness_type soundness_type_; // 사운드니스 타입
size_t RS_extra_dimensions_; // Reed-Solomon 추가 차원
float height_width_ratio_; // 높이-너비 비율
bool make_zk_; // 영지식성 여부
field_subset_type domain_type_; // 도메인 타입
size_t num_constraints_; // 제약 조건 수
size_t num_variables_; // 변수 수
// 계산된 매개변수들
```

```
size_t systematic_domain_dim_; // 시스템 도메인 차원
size_t codeword_domain_dim_; // 코드워드 도메인 차원
size_t query_bound_; // 쿼리 상한
};
```

2. 매개변수 초기화 과정

2.1 도메인 크기 계산

```
// ligero_iop.tcc에서의 도메인 크기 계산
size_t num_vars = num_variables + 1;
size_t systematic_domain_size = (std::size_t) ceil(sqrt(num_vars / this->height_
systematic_domain_size = libff::round_to_next_power_of_2(systematic_domain_size)
this->systematic_domain_dim_ = libff::log2(systematic_domain_size);
this->codeword_domain_dim_ = this->systematic_domain_dim_ + this->RS_extra_dimen
```

핵심 설계 원리:

1. **변수 수 기반 계산**: 총 변수 수에서 시스템 도메인 크기 도출
2. **높이-너비 비율 적용**: `height_width_ratio_` 를 사용하여 행렬 차원 균형 조정
3. **2의 거듭제곱 정렬**: FFT 효율성을 위해 도메인 크기를 2의 거듭제곱으로 조정
4. **Reed-Solomon 확장**: 추가 차원으로 코드워드 도메인 확장

2.2 사운드니스 매개변수 설정

```
void ligero_iop_parameters<FieldT>::set_soundness_parameters()
```

```
{
// 4가지 사운드니스 오류 소스에 대한 균등 할당
const size_t interactive_soundness_bits = this->security_parameter_ + 2;
const size_t query_soundness_bits = this->security_parameter_ + 2;
// 각 구성 요소별 2^{-security_parameter - 2} 오류 할당
}
```

사운드니스 오류 분석:

1. **Direct LDT 쿼리 오류**: 저차 테스트에서의 쿼리 기반 오류
2. **LDT Reducer 상호작용 오류**: 리듀서 프로토콜의 상호작용 오류
3. **인코딩된 프로토콜 상호작용 오류**: R1CS 인코딩 프로토콜의 상호작용 오류
4. **인코딩된 프로토콜 쿼리 오류**: 근접성 매개변수 기반 쿼리 오류

높이-너비 비율(height_width_ratio) 조정의 영향

1. 비율 설정의 수학적 배경

```
// 기본값 설정 (profiling 코드에서)
float height_width_ratio = 0.1; // 기본값: 너비가 높이의 10배
```

2. 행렬 차원 계산에 미치는 영향

2.1 행렬 구성 과정

```
void configure_encoded_ligero_params(const size_t num_variables, const size_
```

```
{

size_t num_vars = num_variables + 1;

const  size_t systematic_domain_size = 1ull << this->systematic_domain_dim_;

// 너비 계산: 변수를 행렬 너비에 맞게 분할

this->num_oracles_input_ = (size_t) ceil(((float) num_vars) / systematic_domain_

std::size_t matrix_width = systematic_domain_size * this->num_oracles_input_;

// 높이 계산: 제약 조건을 행렬 높이에 맞게 분할

std::size_t matrix_height = num_constraints;

if (matrix_height % systematic_domain_size != 0) {

matrix_height += systematic_domain_size - matrix_height % systematic_domain_size

}

this->num_oracle_vectors_ = matrix_height / systematic_domain_size;

}
```

2.2 비율별 성능 특성 분석

| height_width_ratio | 행렬 형태 | 장점 | 단점 |

|---|---|---|---|

| 0.1 (낮은 비율) | 넓고 낮은 행렬 | • 적은 쿼리 수
• 빠른 검증 | • 많은 RS 인코딩
• 높은 메모리 사용 |

| 1.0 (정사각형) | 정사각 행렬 | • 균형잡힌 성능
• 중간 복잡도 | • 중간 수준의 모든 비용 |

| 10.0 (높은 비율) | 좁고 높은 행렬 | • 적은 인코딩 오버헤드
• 낮은 메모리 사용 | • 많은 쿼리 필요

- 느린 검증 |

3. 최적화 전략

3.1 동적 비율 조정

```
// 회로 크기에 따른 적응적 비율 설정 (의사코드)
float optimal_height_width_ratio(size_t num_constraints, size_t num_variables
float circuit_density = (float)num_constraints / num_variables;
if (circuit_density > 2.0) {
return 0.05; // 밀집 회로: 더 넓은 행렬
} else if (circuit_density < 0.5) {
return 0.5; // 희소 회로: 더 높은 행렬
} else {
return 0.1; // 기본값
}
}
```

3.2 메모리 제약 고려

```
// 메모리 사용량 추정
size_t estimate_memory_usage(size_t matrix_width, size_t matrix_height, size_
return matrix_width * matrix_height * expansion_factor * sizeof(FieldT);
}
```

RS_extra_dimensions와 보안성의 관계

1. Reed-Solomon 확장 차원의 역할

```
// 코드워드 도메인 크기 계산
this->codeword_domain_dim_ = this->systematic_domain_dim_ + this->RS_extra_dimen
```

확장 차원의 보안적 의미:

- **최소 거리 증가**: 코드의 최소 거리가 `2^RS_extra_dimensions` 만큼 증가
- **오류 정정 능력**: 더 많은 오류를 감지하고 정정 가능
- **근접성 테스트 강화**: 저차 다항식 검증의 정확도 향상

2. 보안성과 효율성의 트레이드오프

2.1 RS_extra_dimensions = 1인 경우

```
// 최소 확장: 2배 확장
size_t codeword_size = systematic_size * 2;
```

특성:

- **장점**: 최소 오버헤드, 빠른 인코딩/디코딩
- **단점**: 낮은 오류 정정 능력, 약한 보안성

2.2 RS_extra_dimensions = 2인 경우 (기본값)

```
// 표준 확장: 4배 확장
```

```
size_t codeword_size = systematic_size * 4;
```

특성:

- **장점**: 균형잡힌 보안성과 효율성
- **단점**: 중간 수준의 오버헤드

2.3 RS_extra_dimensions ≥ 3인 경우

```
// 고보안 확장: 8배 이상 확장
size_t codeword_size = systematic_size * (1 << RS_extra_dimensions);
```

특성:

- **장점**: 높은 보안성, 강력한 오류 정정
- **단점**: 높은 계산 및 저장 오버헤드

3. 동적 차원 조정 메커니즘

```
// ligero_iop.tcc에서의 동적 조정
if (max_poly_degree > (1ull << this->codeword_domain_dim_)) {
printf("Query bound is too large for codeword domain dimension %lu, \
increasing codeword domain dimension\n", this->codeword_domain_dim_);
this->RS_extra_dimensions_ += 1;
this->codeword_domain_dim_ += 1;
this->set_soundness_parameters();
return;
```

```
}
```

자동 조정 로직:

1. **쿼리 상한 검사**: 현재 도메인 크기로 처리 가능한지 확인
2. **차원 증가**: 필요시 RS_extra_dimensions를 1씩 증가
3. **재계산**: 모든 매개변수를 새로운 차원으로 재계산
4. **반복**: 조건을 만족할 때까지 반복

4. 근접성 매개변수와의 상호작용

```
void  calculate_encoded_ligero_proximity_parameters(const  size_t  query_bound)
// 최소 절대 근접성 매개변수 계산
const  size_t min_absolute_proximity_parameter =
((1ull << this->codeword_domain_dim_) - 2*(1ull << this->systematic_domain_dim_)
- 2*query_bound + 1) / 4 - 1;
// LDT 리듀서 출력과 비교하여 최소값 선택
if (this->ldt_reducer_params_.absolute_proximity_parameter() <= min_absolute_pro
this->absolute_encoded_ligero_proximity_parameter_ =
this->ldt_reducer_params_.absolute_proximity_parameter();
} else {
this->absolute_encoded_ligero_proximity_parameter_ = min_absolute_proximity_para
}
}
```

근접성 매개변수 최적화:

1. **상한 계산**: 코드워드 도메인 크기 기반 상한 설정
2. **LDT 연동**: LDT 리듀서의 근접성 매개변수와 조화
3. **보안성 보장**: 이론적 보안 요구사항 만족
4. **효율성 최적화**: 가능한 한 큰 근접성 매개변수 사용

쿼리 복잡도 최적화

1. 영지식성 여부에 따른 쿼리 계산

1.1 비영지식 경우 (make_zk = false)

```
if (!this->make_zk_) {
const size_t query_bound = 0; // 블라인딩 없음
const size_t query_error_numerator = 2*(1ull << this->systematic_domain_dim_) -
const long double query_error = ((long double) query_error_numerator) / codew
// 단순한 쿼리 수 계산
const long double error_per_query = log2l(query_error + fractional_proximity_p
const long double num_query_repetitions = ceil(-1.0 * query_soundness_bits / e
}
```

1.2 영지식 경우 (make_zk = true)

```
if (this->make_zk_) {
// 순환 의존성 해결을 위한 반복적 추정
```

```
size_t estimated_num_queries = 1;

size_t last_num_queries = 0;

while (estimated_num_queries != last_num_queries) {

const  size_t query_bound = estimated_num_queries + 1;

const  size_t query_error_numerator = 2*(1ull << this->systematic_domain_dim_)

+ 2*query_bound - 2;

// 블라인딩을 고려한 쿼리 오류 계산

const  long  double query_error = ((long  double) query_error_numerator) / codew

// 수렴할 때까지 반복

estimated_num_queries = last_num_queries;

last_num_queries = calculated_queries + direct_ldt_queries;

}

}
```

2. 쿼리 오류 분석

2.1 쿼리 오류 공식

```
비영지식: ϵ_q = (2H - 2) / L

영지식: ϵ_q = (2H + 2b - 2) / L


여기서:

- H: 시스템 도메인 크기 (2^systematic_domain_dim)
```

```
- L: 코드워드 도메인 크기 (2^codeword_domain_dim)

- b: 쿼리 상한 (블라인딩 매개변수)
```

2.2 총 쿼리 사운드니스

```
총 오류 = (ϵ_q + ϵ_p)^r


여기서:

- ϵ_q: 쿼리 오류

- ϵ_p: 근접성 매개변수

- r: 쿼리 반복 횟수
```

3. 성능 최적화 기법

3.1 배치 쿼리 처리

```
// 의사코드: 배치 쿼리 최적화

void  process_queries_in_batches(const std::vector<query_position>&  positions)

const  size_t batch_size = 64; // 캐시 라인 크기 고려

for (size_t i = 0; i < positions.size(); i += batch_size) {

size_t end = std::min(i + batch_size, positions.size());

process_query_batch(positions.begin() + i, positions.begin() + end);

}

}
```

3.2 메모리 접근 패턴 최적화

```
// 행렬 접근 최적화
template<typename FieldT>
void  optimized_matrix_access(const std::vector<std::vector<FieldT>>&  matrix,
const std::vector<size_t>&  row_indices,
const std::vector<size_t>&  col_indices) {
// 행 우선 접근으로 캐시 효율성 향상
std::sort(row_indices.begin(), row_indices.end());
for (size_t row : row_indices) {
for (size_t col : col_indices) {
process_element(matrix[row][col]);
}
}
}
```

실제 구현에서의 최적화 사례

1. 템플릿 기반 필드 추상화

```
template<typename FieldT>
class ligero_iop_parameters {
// 다양한 유한체에 대한 일반화된 구현
// 컴파일 타임 최적화 가능
```

```
};
```

2. 도메인 핸들 시스템

```
// 효율적인 도메인 관리
domain_handle codeword_domain_handle_;
domain_handle systematic_domain_handle_;
domain_handle extended_systematic_domain_handle_;
```

3. 오라클 핸들 벡터화

```
// 벡터화된 오라클 처리
std::vector<oracle_handle_ptr> w_vector_handles_;
std::vector<oracle_handle_ptr> concatenated_vector_handles_;
```

이러한 구현 분석을 통해 Ligero 프로토콜이 이론적 효율성을 실제 구현에서도 달성하기 위해 사용하는 다양한 최적화 기법들을 확인할 수 있습니다.